```
  # now redefine process and delegate to original_process
  def process(action, params=nil, session=nil, flash=nil,
http_method='GET')
    params = Hash[*params.map {|k, v| [k, v.to_s]}.flatten]
    original_process(action, params, session, flash, http_method)
  end
end
```

That's the method `get`, `post`, etc., delegate the work to.

That technique has a risk, it could be the case that `:original_process` was taken. To try to avoid collisions people choose some label that characterizes what the chaining is about:

```
ActionController::TestCase.class_eval do
  def process_with_stringified_params(...)
    params = Hash[*params.map {|k, v| [k, v.to_s]}.flatten]
    process_without_stringified_params(action, params, session, flash,
http_method)
  end
  alias_method :process_without_stringified_params, :process
  alias_method :process, :process_with_stringified_params
end
```

The method `alias_method_chain` provides a shortcut for that pattern:

```
ActionController::TestCase.class_eval do
  def process_with_stringified_params(...)
    params = Hash[*params.map {|k, v| [k, v.to_s]}.flatten]
    process_without_stringified_params(action, params, session, flash,
http_method)
  end
  alias_method_chain :process, :stringified_params
end
```

Rails uses `alias_method_chain` all over the code base. For example validations are added to `ActiveRecord::Base#save` by wrapping the method that way in a separate module specialized in validations.

Defined in `active_support/core_ext/module/aliasing.rb`.

3.2 Attributes

3.2.1 `alias_attribute`

Model attributes have a reader, a writer, and a predicate. You can alias a model attribute having the corresponding three methods defined for you in one shot. As in other aliasing methods, the new name is the first argument, and the old name is the second (one mnemonic is that they go in the same order as if you did an assignment):

```
class User < ActiveRecord::Base
  # You can refer to the email column as "login".
  # This can be meaningful for authentication code.
  alias_attribute :login, :email
end
```

Defined in `active_support/core_ext/module/aliasing.rb`.

3.2.2 Internal Attributes

When you are defining an attribute in a class that is meant to be subclassed, name collisions are a risk. That's remarkably important for libraries.

Active Support defines the macros `attr_internal_reader`, `attr_internal_writer`, and `attr_internal_accessor`. They behave like their Ruby built-in `attr_*` counterparts, except they name the underlying instance variable in a way that makes collisions less likely.

The macro `attr_internal` is a synonym for `attr_internal_accessor`: